

St. Cloud State University

Team Members: Benjamin Pratt, Moez Iqbal, Colton Peterson, Evan Kasseth, McKenna Buckley, Said Mohaddes Sadeqi, Jasper McLean

9/6/2026

PROJECT PROPOSAL

D2L Brightspace Content Development Kit

I. PROJECT BACKGROUND	2
II. PROJECT GOALS	3
III. INITIAL INVESTIGATIONS	5
IV. PROJECT SCOPE	7
V. EVALUATION	13
VI. NEXT STEPS	14

I. Project Background

SUMMARY

Our team is a seven-member software engineering capstone team at St. Cloud State University in the Department of Computing, Informatics, and Data Sciences. Our project aims to build upon an existing toolkit to allow a faculty member to build, test, and deploy interactive, accessible content inside Brightspace without writing code or working with an AI agent directly. This will allow any faculty member, regardless of software development experience, to build and deploy safe, customizable content that lives directly inside of their D2L course.

PROBLEM STATEMENT

Faculty at SCSU are limited to whatever Brightspace's built-in tools offer directly (text, files, and quizzes, etc...). Building anything genuinely interactive means choosing one of four structurally different ways D2L allows custom content into a course: an externally hosted page, an HTML/JavaScript content topic, a SCORM package, or a homepage widget. Each behaves differently; a content topic can tell who's viewing it but can't record a grade, while a SCORM package can record a grade automatically but has no idea who the viewer is. Choosing correctly among these, and then actually building one, requires web development skills most faculty do not have and will not acquire. This is a real technical barrier that keeps a good pedagogical idea from ever getting built, no matter how promising it is. Separately, WCAG 2.2 accessibility conformance is now required for all course material, and remediation is repetitive, rule-driven work nobody on campus has automated yet.

A working prototype of the toolkit that solves the technical half of this problem already exists: it measured exactly what SCSU's Brightspace tenant allows and built a set of AI-agent tools that pick the right avenue for it to be constructed. Operating it still means typing instructions straight to an AI agent, the way a programmer would, and it has never been tested on anyone without that comfort. This capstone exists to close that gap, so that any faculty member (regardless of technical background) can add optional, custom interactive content to their own course whenever they want to. Technical skill should never be the reason that content doesn't get built.

II. Project Goals

PROJECT GOALS

- **Harden & Document:**
 - Close current defects already identified within the prototype's own finding log. Bring documentation up to date with the current kit and produce robust documentation for the end-kit.
- **Front End / Back End:**
 - Build a local browser configurator and preview in front of a hardened back end, so an instructor never interacts with an AI agent directly. Includes a small starter-template library so a build can start from something concrete instead of a blank request, and an optional advanced mode that exposes finer-grained tooling for technical users without exposing it to everyone else by default.
- **Pedagogical Layer:**
 - Add an automated Transparency in Learning and Teaching (TILT) and Universal Design for Learning (UDL) check that flags assignment clarity and design gaps back to the instructor through the front end. The instructor can choose to proceed past a warning. The point is visibility, not a hard gate.
- **Accessibility Enforcement:**
 - Add real accessibility rules to the existing lint/QA gate so a build that fails contrast, alt-text, keyboard, or heading-structure checks does not ship. Pair this with a plain-language accessibility report for the instructor, not pass/fail, explaining what's wrong and how to fix it. Add additional WCAG 2.2 accessibility checks to the lint harness.
- **Multi-Language Generation:**
 - Since the AI agent is already generating the content, generate a second-language version of the same build on request, rather than treating translation as a separate manual step.
- **Deployment & Agent Flexibility:**
 - Design the system so it can run either locally, on an instructor's own machine, or on a hosted, IT-managed VM if that's approved. One back end, not two separate products to maintain.
- **Validation:**
 - Prove the result works for a genuine first-time faculty user, and not just for the person who builds it.

PROJECT ADVANTAGES & MEASURABLE OBJECTIVES

Objective	Target Metric	Measurement Method
Novice usability	An instructor with no prior exposure to the kit completes a first build and deployment unassisted.	Observed usability session, compared directly against the original pilot's inferences.
Defect & documentation closure	Zero regressions across the existing test suite after every hardening fix.	Automated lint and emulator regression suite, run on every change. Documentation cross-check
Agent independence	100% of a build's steps are completed through the front end, with no direct agent interaction.	End-to-end configure → build → deploy demonstration.
TILT/UDL flag usefulness	Instructor rated: on a sample of real pilot assignments, at least 80% of TILT/UDL flags raised are judged by the instructor as accurate and worth acting on. These must not be false positives.	Automated check run against a sample of real pilot assignments, compared to instructor judgment.
Accessibility enforcement	100% of rebuilt activities must pass the accessibility lint rules before deployment. Manual auditing should present zero WCAG 2.2 violations on the builds sampled.	Automated lint suite plus a manual accessibility audit on a sample of builds.
Multi-language output quality	A fluent speaker rates at least 90% of sampled translations as accurate and usable without any editing. This means no preserved or broken formatting.	Manual review of generated non-English content by a fluent speaker, on a sample of builds
Deployment flexibility	The same back-end runs, unmodified, in both a local and a hosted-VM configuration.	Full build/deploy flow demonstrated in each environment.

III. Initial Investigations

STAKEHOLDERS, KEY PERSONNEL, & TARGET USERS

- Project Sponsor: Mark Gill
- Course Advisor & Instructor: Dr. Omar Al-Azzam
- Engineering Team: Benjamin Pratt, Moez Iqbal, Colton Peterson, Evan Kasseth, McKenna Buckley, Jasper McLean, Said Mohaddes Sadeqi
- Target End Users: SCSU Faculty

TIMELINE & MILESTONE SCHEDULE (SE490 & SE491)

Phase	Term & Module	Milestones & Target Deliverables
Setup & Hardening	SE490 (Modules 1–2)	Finalize proposal, set up the GitHub Projects backlog, close the packaging and documentation defects already diagnosed in the prototype, and confirm a clean regression baseline.
Core Fixes, Accessibility Rules & Front-End/Back-End Foundation	SE490 (Modules 3–5)	Add the first accessibility rules to the lint/QA gate; scaffold the local server and configurator UI, including an initial starter-template option; define the front-end/back-end API boundary so hosting target (local vs. VM) and AI agent (Claude vs. Copilot) are configuration choices from the start, not hard-coded assumptions
Integration	SE490 (Modules 6–7)	Wire the TILT/UDL check end to end; demo the full configure → build → deploy flow through the front end against the hardened Claude back end; SE490 code freeze.
Testing Prep & Depth	SE491 (Modules 1–3)	Run Tier 1 synthetic-persona testing against real assignment materials; recruit a novice faculty volunteer; build 1–2 additional avenue examples and flesh out the starter-template library; begin multi-language generation on a sample build.
Novice Validation	SE491 (Modules 4–5)	Run the unassisted novice faculty usability test; fold findings back into the kit and the front end; review multi-language output quality with a fluent speaker.
Copilot Phase, Robust Documentation, & Hand-off	SE491 (Modules 6–7)	Attempt back-end integration with Copilot and document any parity gap; deploy the same back end to the hosted VM if it's been approved by then, confirming the flexible design from Phase 2 actually holds; add the optional power-user/advanced mode if time allows; finalize documentation; final report, demo, and hand-off.

PROJECT BUDGET

Because our project uses existing hardware, free developer tooling, and sponsor-provided infrastructure, most of the direct budget is \$0.00. Two items depend on decisions still pending with the Mark and SCSU IT and are disclosed here rather than assumed away.

- **Hosting & Compute:** Local machines only, if the kit stays local (\$0.00). If a hosted VM is approved through IT instead, the university would host and maintain it. Cost and ownership are not yet determined and are pending that approval.
- **D2L Sandbox & Faculty Volunteer Time:** Provided by Mark Gill (\$0.00 to the team).
- **Development Tools:** GitHub, VS Code, and similar tools at their free tier (open source, \$0.00).
- **AI Agent Usage (Claude now, Copilot in a later phase):** If the hosted-VM path is approved and runs through a single Anthropic API instead, that introduces a real, ongoing API cost at a scale this project does not commit to estimating.

RISK MANAGEMENT

Risk Identified	Impact	Probability	Mitigation Strategy
Known hardening defects take longer than the scheduled window allows	High	Medium	Both are explicitly scheduled in Modules 1–5 of the timeline, before new example builds or parallel team development depend on the QA gate covering them; if either slip, no new avenue work starts until it's closed.
Accessibility rules are genuinely harder to get right than the kit's existing mechanical rules	High	Medium	Start from WCAG 2.2's own success criteria, prioritizing ones that are mechanically checkable like the existing 22 rules (alt text, contrast, heading structure) before harder judgment-based ones. Each new rule ships with its own regression test, same as the existing 22, and pairs with the plain-language accessibility report so both land together.
Deployment & Agent Flexibility adds real design complexity, and the VM half depends on an approval outside the team's control	Medium	Medium	Build one back end with hosting location and AI agent as configuration choices, not two separate products; the local path ships regardless of what IT decides, and VM/Copilot support get verified when ready rather than blocking the core plan.
Novice faculty recruitment slips past the test window	High	Medium	Start recruiting in SE491 Module 1, well ahead of the Module 4–5 test window; Mark already has faculty relationships to draw on.

7-person team coordination	Medium	High	Track weekly work in GitHub Projects, and split into front-end, back-end, and QA sub-teams
----------------------------	--------	------	--

IV. Project Scope

INITIAL CONTEXT MODEL

Our system operates as one unit that sits between the instructor and Brightspace, with the AI agent as an internal implementation detail rather than something the instructor touches directly.

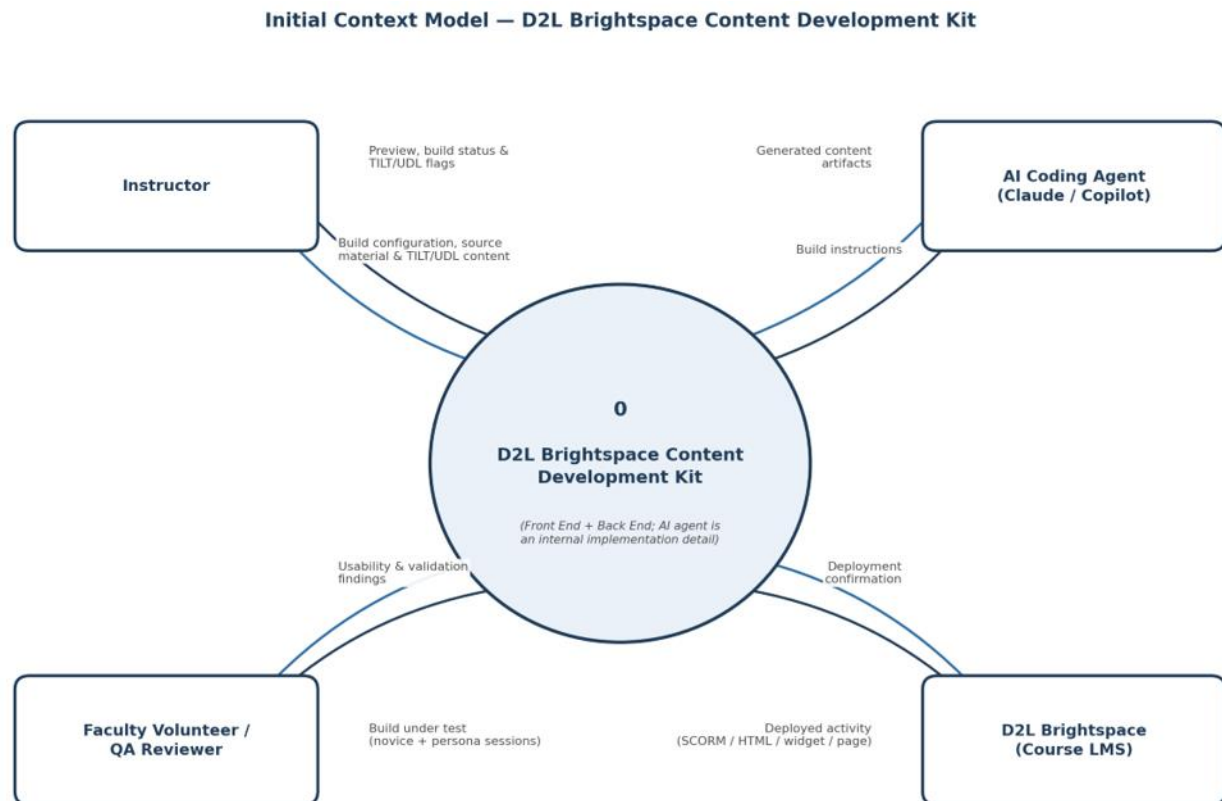


Figure 1: Initial context model (Level 0 DFD): the system as a single process, showing the four external entities, along with incoming and outgoing dataflows. Students are not a separate external entity as they never interact with the kit directly, only the activity that is generated and deployed into the D2L Brightspace, which is the boundary shown in the figure.

INCOMING DATA FLOWS

- **Instructor:** The instructor tells the system what type of content they would like to produce, gives necessary source materials and other assignment data like build configuration (content type, source material) and assignment content submitted for the TILT/UDL check, through the front end.
- **AI agent:** AI delivers generated content artifacts returned to the back end for QA gating before deployment.
- **D2L Brightspace:** Confirmation of successful completion and deployment is delivered back to the system, so the team can check if the content has been successfully included in the course.
- **The Faculty Volunteer / QA Reviewer:** the build under test, for the novice usability and output language quality review.

OUTGOING DATA FLOWS

- **Instructor:** The rendered preview, plain language build status, and TILT/UDL flags.
- **The AI agent:** build instructions dispatched by the back end.
- **D2L Brightspace:** The system delivers final activity or content package for inclusion into the instructor's course. The deployed activity (SCORM / HTML / widget / page).
- **Faculty Volunteer / QA Reviewer:** The system delivers a final activity that is to be tested by the faculty member.

INTERNAL SYSTEM SCOPE & BUSINESS EVENTS

- **Configuration & content check:** the instructor submits a build configuration through the front end; the back end automatically runs the TILT/UDL check and returns flags before the build proceeds.
- **Content generation:** the back-end dispatches build instructions to Claude (Copilot in a later phase); the agent generates content per the existing skill guides.
- **Quality gate:** the lint and emulator suite runs automatically; any error halts deployment and returns a plain-language finding instead of a stack trace.
- **Deployment & verification:** the back end deploys to D2L and reads back the result to confirm what was actually recorded, rather than trusting a successful response code alone.
- **Validation:** a novice faculty usability session and a Tier 1 synthetic-persona pass generate findings that feed back into the product backlog.

INITIAL USE-CASE MODEL

Initial Use-Case Model — D2L Brightspace Content Development Kit

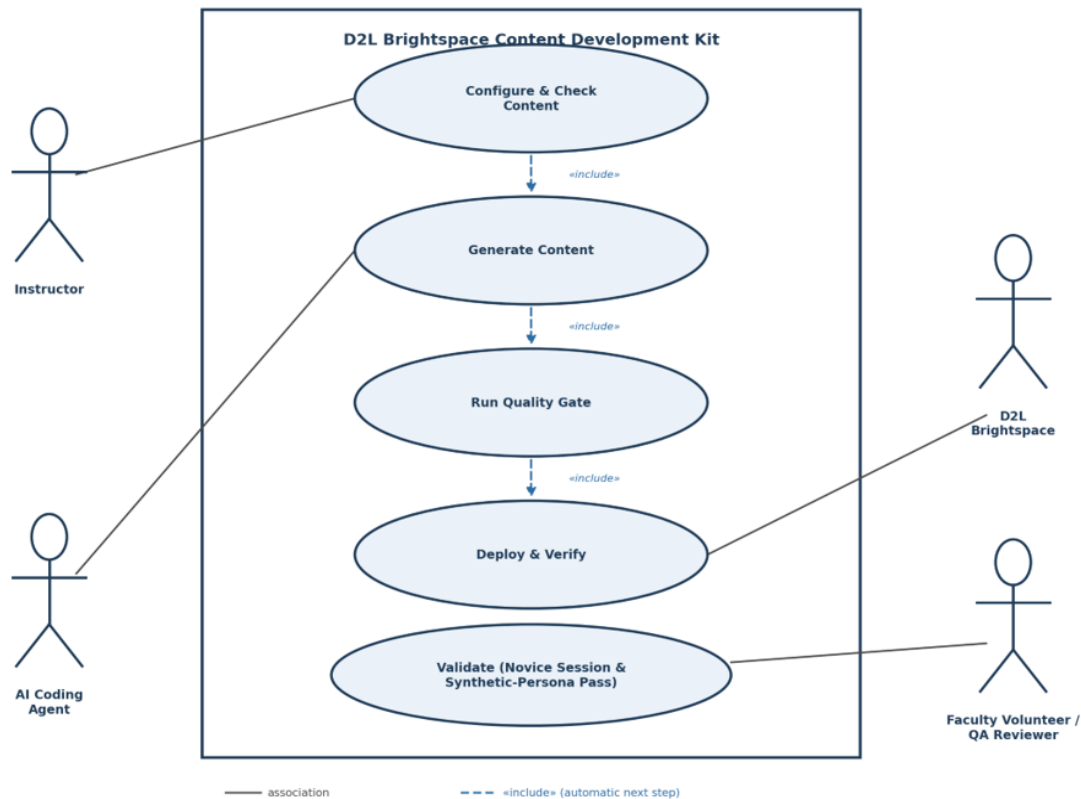


Figure 2: initial use case model

INITIAL USE-CASE MODEL DESCRIPTION

Figure 2 illustrates the key interconnections between the users, external systems, and the D2L Brightspace Content Development Kit. First, the user provides the information necessary to generate an activity, which results in sending corresponding instructions to the AI coding agent that produces the content requested.

After the content generation, it goes through technical and accessibility checks, which are needed for deployment. If the content passes these checks, it is then delivered to D2L Brightspace and checked for deployment.

There are five main use cases, which are Configure & Check Content, Generate Content, Run Quality Gate, Deploy & Verify, and Validate. Configure & Check Content, Generate Content, Run Quality Gate, and Deploy & Verify are part of the main process of content development. After configuration, most of the process can take place automatically, without involving the instructor in the control of the AI agent.

Using a case called Validate is separate from others, as it concerns evaluation of the system and not the development of an activity. Volunteers and QA reviewers can evaluate the developed system, and the outcomes of this process may be entered into the product's backlog.

PRODUCT BACKLOG MANAGEMENT

To manage development using agile processes, our team will maintain and update the product backlog in GitHub Projects (repo: github.com/Wolftank/D2L-Brightspace-Content-Development-Kit). Requirements are broken into high-level epics, refined weekly into prioritized sprint backlog items. We will start with Sprint 1 in Module 2 of SE490.

EPIC BREAKDOWN

1. **Hardening & Defect Closure:** Fix the packaging gap, the SCORM emulator's credit-mode blind spot, the filename-collision risk, and the router/documentation inconsistencies already identified in the prototype's findings log.
2. **Front-End Configurator & Preview:** Build the local server, the browser-based build configuration UI, and a preview that mirrors D2L's real rendering constraints.
3. **Back-End Orchestration API:** Define and harden the internal API that drives the AI agent, the QA gate, and D2L deployment, starting with Claude.
4. **TILT/UDL Pedagogical Check:** Implement the automated backend pass that flags assignment clarity and accessibility gaps, reported through the front end.
5. **Novice & Synthetic-Persona Validation:** Build the Tier 1 persona-testing harness and run the unassisted novice faculty usability test.
6. **Avenue Depth & Copilot Adoption:** Build deeper example activities across existing avenues and attempt Copilot integration once the Claude path is solid.

Initial Product Backlog Items

The team plans to implement the initial product backlog, which includes the features and upgrades during SE490 and SE491. These will be managed using GitHub Projects and may evolve as the team gains insight into the system.

PB-01: As an instructor, I want to create an activity via a browser interface so that I do not have to use the command line.

PB-02: As an instructor, I want to choose a starter template so that I have a clear starting point while creating new content.

PB-03: As an instructor, I want to preview generated content before deployment so that I can examine how it looks and works.

PB-04: As an instructor, I want the system to point out TILT/UDL issues so that I can make my assignments more understandable and clear.

PB-05: As an instructor, I want generated content to be verified for accessibility so that content that has significant accessibility issues will not be deployed.

PB-06: As an instructor, I want to have accessibility issues explained clearly so that I will be able to understand what is wrong.

PB-07: As an instructor, I want the system to determine the correct Brightspace content type so that I would not have to understand the technical difference between SCORM, HTML, widgets, and hosted pages.

PB-08: As an instructor, I want the option to generate the same content in another language so that it could help even more students.

PB-09: As an instructor, I want to be able to deploy content directly to D2L using the toolkit so that the process does not become complicated.

PB-10: As a team member, I want the system to verify each D2L deployment so that we can be sure that activity was actually added.

PB-11: As a team member, I want to have automated tests run after changes so that new development will not break anything.

PB-12: As a faculty tester, I want to finish the whole process on my own so that we can determine if toolkit is feasible for nontechnical instructors.

SPRINT 1 BACKLOG — SE490 (Module 2)

1. Create the initial GitHub Projects Kanban board and populate it from the 7 epics drafted by the team lead.
2. Agree on rules of engagement and a process for merging code into the main tree.
3. Fixup missing references in the d2l-activity-deployer llm skill file.
4. Fix outdated information from the prototype.
5. Fix the router/widget documentation mismatch.
6. Setup an MVP for the local backend.
7. Setup minimal placeholder configurator frontend.

SPRINT 2 BACKLOG — SE490 (MODULES 3–5)

1. Establish the connection between the front-end interface and the back-end system.
2. Add the first WCAG 2.2 accessibility checks to the existing linting system.
3. Create automated tests for each new accessibility rule.
4. Develop a basic accessibility report that explains problems in language an instructor can understand.
5. Add starter template options to the browser configurator.
6. Connect the configurator to the existing Claude-based content-generation process.
7. Develop an early preview feature that allows instructors to examine generated content before deployment.
8. Continue updating technical and user documentation as the system develops.

SPRINT 3 BACKLOG — SE490 (MODULES 6–7)

1. Integrate the TILT/UDL review into the complete content development process.
2. Display TILT/UDL warnings and suggestions through the front-end interface.
3. Complete the full workflow from content configuration through generation, testing, previewing, deployment, and verification.
4. Test the system using multiple types of content supported by D2L Brightspace.
5. Identify and correct major problems discovered during integration testing.
6. Prepare a working demonstration of the SE490 version of the project.
7. Review the remaining work and update the product backlog for development during SE491.

V. Evaluation

Project success will be evaluated throughout development using a combination of automated testing, manual review, usability testing, and stakeholder feedback. Evaluation will occur continuously during development rather than only at the end of the project.

Automated lint and regression tests will be run as changes are made to ensure new development does not introduce regressions. Accessibility checks will also be incorporated into the QA process, with completed activities required to pass the implemented accessibility rules before deployment. Selected builds will also receive manual accessibility review against WCAG 2.2 requirements.

Usability will be evaluated through an observed session with a faculty member who has no previous experience with the toolkit. The goal is for the faculty member to successfully

configure, build, and deploy content without assistance or direct interaction with the AI agent. Findings from this session will be recorded and added to the project backlog for improvement.

The TILT/UDL feature will be evaluated using assignment samples and instructor feedback, with a target of at least 80% of generated flags being considered accurate and useful. Multi-language output will be reviewed by a fluent speaker, with a target of at least 90% of sampled translations being usable without editing. Deployment flexibility will be evaluated by demonstrating the same back end in both local and hosted environments if the hosted VM is approved.

The engineering team will be responsible for ongoing technical evaluation and regression testing. The project sponsor, course advisor, and other appropriate reviewers will provide feedback during usability testing and final validation. Evaluation results will be used to prioritize remaining backlog items and determine whether the project's measurable objectives have been met.

VI. Next Steps

Following approval of this project proposal, the team will begin Sprint 1 development during Module 2 of SE490. The immediate next step will be converting the project epics into actionable items within the GitHub Projects board and assigning initial responsibilities across the front-end, back-end, and QA tracks.

The team will then focus on establishing a stable development baseline by resolving the known packaging and documentation defects already identified in the existing prototype and running the current regression suite. At the same time, initial development will begin on the local server and browser-based configurator that will eventually allow faculty to use the toolkit without interacting directly with an AI agent.

Throughout development, the team will review and prioritize the backlog weekly, track progress through GitHub Projects, and adjust work based on testing results and sponsor feedback. Later phases will expand the accessibility checks, integrate the TILT/UDL review, complete the end-to-end build and deployment workflow, and validate the finished system with a first-time faculty user.

At the end of the project, the team will provide the completed toolkit, updated documentation, testing results, and final demonstration to the project sponsor and course advisor for review and hand-off.